Department of Electrical Engineering
Indian Institute of Technology Kharagpur
Machine Learning for Signal Processing Laboratory
(EE69210)
Spring, 2023-24

Experiment 7:
Polynomial Regression

Grading Rubric

	Tick the best applicable per row			Points
	Below Expectation	Lacking in Some	Meets all Expectation	
Completeness of the report				
Organization of the report (5 pts)				
Quality of figures (5 pts)				
Dataset preparation and explanation (10 pts)				
feature scaling and train-test split and explanation (25 pts)				
Implementation of Gradient Descent algorithm and explanation (40 pts)				
Model Evaluation and Inference (15 pts)				
			TOTAL (100 pts)	

1 Code of Conduct

Students are expected to behave ethically both in and out of the lab. Unethical behaviour includes, but is not limited to, the following:

- Possession of another person's laboratory solutions from the current or previous years.
- Reference to, or use of another person's laboratory solutions from the current or previous years.
- Submission of work that is not done by your laboratory group.
- Allowing another person to copy your laboratory solutions or work.
- Cheating on quizzes.

The rules of laboratory ethics are designed to facilitate these goals. We emphasize that laboratory TAs are available to help the student understand the basic concepts and answer the questions asked in the laboratory exercises. By performing the laboratories independently, students will likely learn more and improve their performance in the course as a whole. Please note that it is the student's responsibility to ensure that the content of their graded laboratories is not distributed to other students. If there is any question as to whether a given action might be considered unethical, please see the professor or the TA before you engage in such actions.

2 Theory of Polynomial Regression

Polynomial regression is a type of regression analysis used to model the relationship between the independent variable x and the dependent variable y as an n-th degree polynomial function. Generally x can be a multi-dimensional vector but in this experiment we consider x to be a scalar value. It is particularly useful when the relationship between the variables is nonlinear, and a linear model is insufficient to capture the underlying pattern in the data.

The polynomial regression model can be represented as follows:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_n x^n \tag{1}$$

where each row of matrix $X \in \mathbb{R}^{m \times n}$ represents a unique sample value and $\Theta \in \mathbb{R}^{n \times 1}$ represents the parameters vector.

- $\hat{y}$ is the predicted value,
- x is the data sample,
- $\theta_0, \theta_1, \ldots, \theta_n$ are the coefficients of the polynomial terms,
- n is the degree of the polynomial,

In the matrix form same can be shown as follows:

$$\hat{Y} = \mathbf{X} \cdot \Theta$$

where:

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^n \\ 1 & x_2 & x_2^2 & \ldots & x_2^n \\ 1 & x_3 & x_1^3 & \ldots & x_3^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_m & x_m^2 & \ldots & x_m^n \end{bmatrix}_{m \times n}, \Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}_{n \times 1}, \hat{Y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_1 \\ \vdots \\ \hat{y}_m \end{bmatrix}_{m \times 1}, Y = \begin{bmatrix} y_1 \\ y_1 \\ \vdots \\ y_m \end{bmatrix}_{m \times 1}$$

The goal of polynomial regression is to find the coefficients $\theta_0, \theta_1, \ldots, \theta_n$ that minimize the difference between the predicted values and the actual values of the dependent variable y. This is typically done by minimizing a cost function, such as the mean squared error (MSE), which measures the average squared difference between the predicted and actual values across all data points. Apart from MSE many other loss functions like Mean Absolute Error(MAE), Root Mean Square Error (RMSE), Mean Absolute Percentage Error(MAPE), etc. are also used.

Polynomial regression can be visualized by fitting a curve to the data points, where the degree of the polynomial determines the flexibility or complexity of the curve. Higher-degree polynomials can capture more complex patterns in the data but may also lead to overfitting, where the model learns noise in the data rather than the underlying relationship.

Polynomial regression can be implemented using various methods, including ordinary least squares (OLS) estimation, gradient descent optimization.

3 Feature Scaling

Feature scaling is the process of normalizing or standardizing the features of the dataset. In polynomial regression, feature scaling is important because it helps gradient descent converge faster and more reliably.

When the features are on different scales, gradient descent may take longer to converge or may converge to suboptimal solutions. Feature scaling ensures that all features have a similar scale, preventing some features from dominating others during the optimization process.

Common methods of feature scaling include min-max scaling (scaling features to a range between 0 and 1) and standardization (scaling features to have a mean of 0 and a standard deviation of 1).

$$x_{scaled}^{(i)} = \frac{x^{(i)} - \mu^{(i)}}{\sigma^{(i)}} \tag{2}$$

where:

- $x^{(i)}$ is the i^{th} feature vector,
- $\mu^{(i)}$ is the mean of i^{th} feature vector,
- $\sigma^{(i)}$ the standard deviation of i^{th} feature vector,

4 Importance of Train-Test Split

The train-test split is essential for evaluating the performance of a model and preventing overfitting. It involves dividing the dataset into two subsets: one for training the model and the other for testing it.

By training the model on a subset of the data and testing it on another, we can assess how well the model generalizes to unseen data. This helps to detect whether the model has learned the underlying patterns in the data or if it has simply memorized the training data (overfitting).

A common practice is to split the data into a training set (e.g., 70-80% of the data) and a testing set (e.g., 20-30% of the data). The training set is used to train the model, while the testing set is used to evaluate its performance.

The train-test split should be performed randomly to ensure that both sets are representative of the overall dataset. Additionally, techniques such as cross-validation can be employed to further assess the model's performance and ensure its robustness.

5 Theory of Gradient Descent Algorithm

Gradient Descent is an iterative optimization algorithm used to minimize the cost function of a model. In the context of polynomial regression, the cost function typically measures the difference between the predicted values and the actual values, such as the mean squared error (MSE).

The update rule for gradient descent is given by:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

where:

- θ_j are the parameters (coefficients) of the model,
- α is the learning rate, which controls the step size of each iteration,
- $J(\theta)$ is the cost function.

In each iteration, the algorithm updates the parameters θ_j by moving them in the direction opposite to the gradient of the cost function for θ_j. This process continues until convergence, where the algorithm finds the parameters that minimize the cost function.

Gradient Descent is widely used in machine learning and optimization problems due to its simplicity and efficiency. However, choosing an appropriate learning rate is crucial, as too small a value can result in slow convergence, while too large a value can cause oscillations or divergence.

5.1 Gradient descent for polynomial regression

For polynomial regression, the cost function $J(\theta)$ can be defined as the mean squared error (MSE) between the predicted values and the actual values:

$$J(\Theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

where m is the number of training examples, $h_\theta(x)$ is the hypothesis function, and $x^{(i)}$ and $y^{(i)}$ represent the features and labels of the ith training example.

In vectorized form same can be shown as,

$$J(\Theta) = \frac{1}{2m} (Y - \hat{Y})^T \dot{(} Y - \hat{Y})$$

To update the coefficients θ_j using gradient descent, we compute the partial derivative as follows:

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

where $x_j^{(i)}$ represents the jth feature of the ith training example. As we are doing the polynomial regression where x is 1D data point so the jth feature is the jth power of x.

In the vectorized form the same can be represented as,

$$\Theta_{n\times 1} = \Theta_{n\times 1} - \frac{\alpha}{m} [(Y - \hat{Y})_{1\times m} \dot{X}_{m\times n}]^T$$

6 Experiments on Polynomial Regression

6.1 Importing essential libraries

```
#importing libraries
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
```

Listing 1: Code to import essential libraries

6.1.1 Preparation of the dataset and visualization

```
# Set random seed for reproducibility
np.random.seed(42)

# Generate x values
x = np.linspace(-5, 5, 300).reshape((-1,1))

# Generate y values using a combination of sine, cosine, and polynomial
    functions with added noise
noise = np.random.normal(0, 1, x.shape)
y_true = np.sin(2 * x) + 0.5 * x**2 + np.cos(3 * x)
y_noisy = y_true + noise

y=y_noisy.reshape((-1,1))
# Plot the true function and the different models
plt.figure(figsize=(15,7))
plt.scatter(x, y_noisy, label='Noisy Data')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.title('Dataset for Polynomial Regression')
plt.show()
```

Listing 2: Code to prepare the dataset

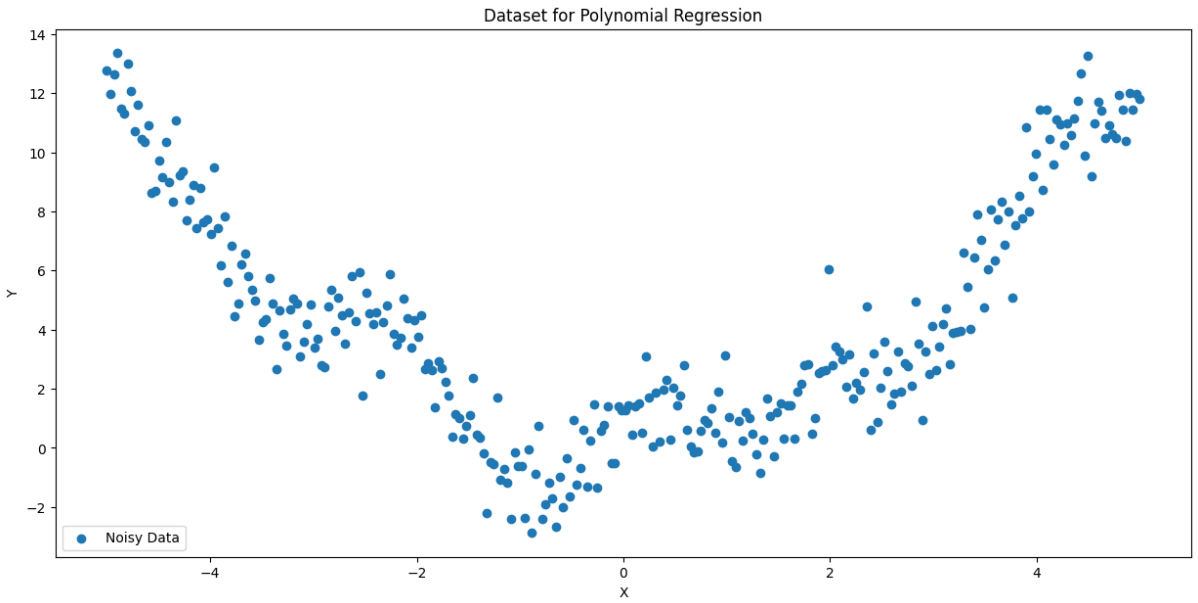

Figure 1: visualization of the dataset.

6.1.2 Assignments to solve and report

Write the codes for the following tasks in a .ipynb file, including all visualizations and submit the executed file. Submit also a separate pdf of your report on this experiment. This report should describe your observations and reasoning while executing these experiments.

1. Rerun all the steps discussed till now, rerunning all the codes. Check your consistency in generating data similar to those presented earlier.
2. Create the matrix X as shown above in the theory for the degree of the polynomial as 2 and split the dataset into train and test data such that 80% of data is used for training and 20% of data is used for testing (It is advised to try out different degrees of the polynomial as well and their effect on the output).
3. Implement the gradient descent algorithm to learn the parameter Θ.
4. Train the model using the gradient descent alogrithm for 500 epochs with Plot the training loss (2).
5. Predict the values of the test set and plot them as shown in figure (3).

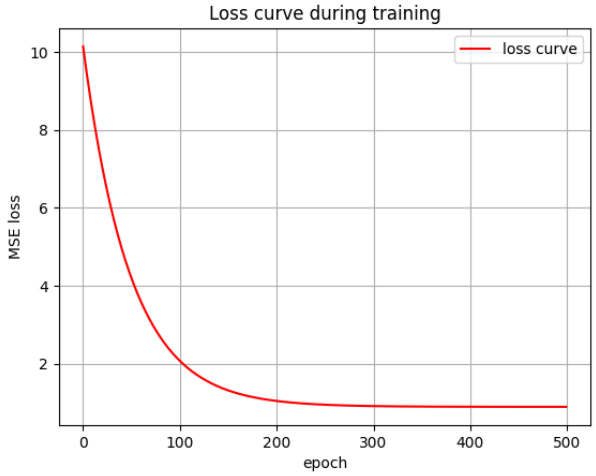

Figure 2: Training loss and accuracy.

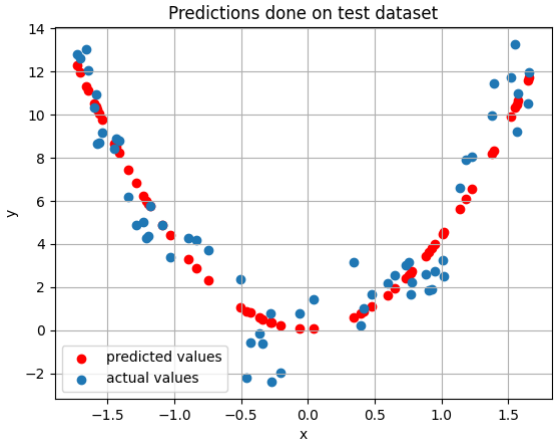

Figure 3: Predictions on the test data.